

CSC8501

Iterated Prisoner's Dilemma Engine

Report and analysis

Kevwe Anaughe
200868305

Design Rationale

I first simplified the steps involved in simulating IPD, to better my own understanding. Identifying that each tournament ran matches between each strategy a set number of times, I decided to use a bottom-up strategy. This meant I began by implementing the functions necessary to run a single match and worked outwards from there. This technique was useful as I usually had a good understanding of the data that would be necessary to pass between translation units.

After defining header files for the tournament and match functionality, I recognised that I would require multiple values from the arguments passed into the program, so rather than passing these values through each class constructor and making unnecessary copies, I opted to using a small global static class.

It was possible a value from this configuration class could be read before it was instantiated, or assigned the correct value, which could lead to unexpected errors. To avoid this issue, I designed an engine class to act as a level of abstraction, obscuring the classes running the tournaments and outputting the data. In the constructor of this engine class, I validate the arguments provided, setting the members of the static config class. Now, as the engine class is responsible for calling the required IPD functions, I can be certain that when the config class is called, it always has the correct values. As an added layer of security, I initialised all members of the config class with default values, inline. This avoids bad read errors in situations where the user does not provide certain arguments.

Starting by writing headers also made me aware of the structures I would require. Throughout my program I rely on the use of unordered maps, as they perform better than standard maps and allow me to store data according to its strategy. To prevent errors, I may have made when referring to strategy names, I used enumerated classes.

However, to store the results from each match, I decided to use a vector of programmer-defined structures. Each structure stored a unique pointer to the object derived from a strategy class for each player. By using unique pointers, I would only instantiate, at most, two objects per strategy used in that tournament. To ensure each repeat of a match begins fresh, I defined a reset method for each class. Moreover, this structure stored the average payoff and confidence intervals for each player. I did this to simplify the process of retrieving this data when outputting the results.

To encapsulate each step in the process of simulating IPD, I used OOP throughout my program. There are many examples of inheritance and polymorphism, most notably in the Agents class. To implement the strategies that could be used in this program, I created a virtual base class that defines the general structure every child class follows. It defined the base decide method that every strategy would have to override. This greatly simplified the process of referring to objects from each strategy, as I could use unique pointers to objects of the base class type instead.

I manually tested my program intermittently during its development. For example, when testing the tournament functionality, I ran the program using predictable strategies, allowing me to predict the results and compare this to the output. In hindsight, I would have plotted graphs from the generated results earlier, rather than using raw data, as this made it much easier to

gauge how reasonable the output was. Towards the end of development, I used an AI agent to write google test files, for automated testing of major classes in my library.

Original Strategies

CAUTIOUS

Cautious takes from the PAVLOV strategy, in which it changes its response in reaction to the payoff from the previous round. However, it differs to PAVLOV as it is initially 'suspicious' of the opponent, but it attempts to apologise for defecting after a set number of consecutive rounds in which it received the punishment payoff. Otherwise, it defaults to using 'win-stay/lose-shift' logic.

Behaviour:

Rounds 1 - 2: Play D for the first two rounds

- This allows it to gain an initial advantage against cooperative strategies
- It plays D for two rounds, to avoid getting stuck in alternating patterns against reactive strategies

Rounds 3+: CAUTIOUS will default to 'win-stay/lose-shift' logic

- If previous outcome was CC (R) or DD (P) -> stays with same choice
- If previous outcome was CD (S) or DC (T) -> switch choice in next round

Rounds 6-7: If the outcomes of the last three rounds were DD, CAUTIOUS will attempt to apologise for defection, by playing C for the next two rounds

- The second round of cooperation is necessary as it allows the opponent to react to the first

Round 8: Returns to 'win-stay/lose-shift' logic

- If reconciliation was successful, both strategies mutually cooperate
- If reconciliation was unsuccessful, both strategies mutually defect

This strategy would benefit most in noisy environments, against other cooperative strategies. If either strategy defects due to noise, it is able to resolve the situation and return to mutual cooperation.

However, it is vulnerable against exploitative strategies because it repeatedly attempts to reconcile with the opponent. In such cases, it is most likely that it will return to mutual defection, but it had to receive the sucker payoff in the process.

Firm But Fair – FBF

FBF initially cooperates with its opponent, until it is betrayed, after which it retaliates proportionally. It maintains a memory of past exploitation to avoid frequent sucker payoffs, but it is able to forgive the opponent after a streak of mutual cooperation. Furthermore, it is able to ignore single rounds of defection that are likely to occur under noise, if it is able to 'trust' the opponent.

Behaviour:

Round 1: Play C

Rule:

- If the previous outcome was CD (S) -> FBF detects betrayal and retaliates
 - It stores the number of times it has been exploited in memory
 - For N betrayals, FBF punishes for N+1 rounds
 - This value has a set maximum to allow for forgiveness (e.g. 5)
 - FBF ignores betrayal if the opponent has a streak of mutual cooperation greater than the forgiveness threshold.
- After punishment, it plays C to signal for cooperation (apology phase)
 - During this phase, it does not count any CD outcomes as betrayal
- After three rounds (forgiveness threshold) of mutual cooperation, it subtracts one from the number of times it has been exploited
- If the opponent cooperates at any point during the match, FBF labels them as trustworthy
 - If the opponent is not trusted, FBF does not attempt to cooperate at all

By operating off trust, FBF can resist exploitation. In noise-free environments, it is expected that most of its matches will result in either mutual cooperation or defection.

When noise is introduced, it is forgiving of mistakes made by the opponent, if they have proven to be trustworthy. However, if FBF makes a mistake due to noise, it can break a mutual cooperation streak and sometimes result in unnecessary punishment. This particular situation increases in likelihood as noise increases, making the strategy less effective.

Assessment Questions

Q1: Build the engine & CLI; run baseline tournament

“Example tournament with all strategies” arguments:

```
--rounds 120 --repeats 12 --epsilon 0.03 --seed 7 --strategies  
ALLC,ALLD,TFT,GRIM,PAVLOV,RND0.3,CONTRITE,PROBER,FBF,CAUTIOUS  
--format text
```

Results stored in “results/Q1 results/Tournament_AllStrats.txt”

I have run a tournament using all the strategies implemented in my project, with very little noise.

The results, displayed on the next page, show that PROBER is able to maximise its average payoff by exploiting and cooperating with its opponents, appropriately. ALLD earns the lowest average payoff, as it is only able to exploit ALLC and RND0.3. Against other strategies in this environment, it is more likely that it will experience mutual defection.

Tournament Summary

Rounds: 120 Repeats: 12 Seed: 7 Epsilon: 0.03
Payoffs: T=5, R=3, P=1, S=0

Leaderboard

Strategy Mean
1. PROBER: 2.613, 95% CI [2.46, 2.765]
2. FBF: 2.476, 95% CI [2.382, 2.571]
3. CONTRITE: 2.413, 95% CI [2.325, 2.501]
4. CAUTIOUS: 2.371, 95% CI [2.263, 2.478]
5. TFT: 2.351, 95% CI [2.265, 2.437]
6. PAVLOV: 2.329, 95% CI [2.242, 2.417]
7. GRIM: 2.209, 95% CI [2.059, 2.36]
8. RND0.3: 2.176, 95% CI [2.028, 2.323]
9. ALLC: 2.15, 95% CI [2, 2.3]
10. ALLD: 2.144, 95% CI [1.966, 2.322]

Payoff Matrix

ALLC ALLD TFT GRIM PAVLOV RND0.3 CONTRITE PROBER FBF CAUTIOUS
ALLC 2.972 0.1097 2.889 0.7274 2.809 2.976 2.93 0.4128 2.92 2.753
ALLD 4.844 1.08 1.204 1.127 1.336 4.787 1.23 1.3 1.979 2.549
TFT 3.016 1.038 2.456 1.352 2.249 3.026 2.61 2.234 2.901 2.63
GRIM 4.432 1.078 1.491 1.396 1.573 4.28 1.568 1.289 2.436 2.55
PAVLOV 3.073 1.036 2.251 1.358 2.211 3.091 2.347 2.3 2.872 2.756
RND0.3 2.959 0.1434 2.892 0.9556 2.782 2.967 2.93 0.3892 2.951 2.788
CONTRITE 2.994 1.06 2.565 1.413 2.282 2.994 2.817 2.37 2.91 2.724
PROBER 4.626 1.043 2.26 1.131 2.31 4.655 2.435 2.249 2.803 2.612
FBF 3.002 0.8559 2.812 1.396 2.634 2.984 2.852 2.588 2.958 2.682
CAUTIOUS 3.115 0.7247 2.566 0.741 2.639 3.099 2.757 2.567 2.816 2.682

“Baseline round-robin tournament” arguments:

```
--rounds 150 --repeats 10 --seed 7 --strategies  
ALLC,ALLD,TFT,GRIM,PAVLOV --format text
```

Results stored in “results/Q1 results/BaselineTournament.txt”

Tournament Summary

```
-----  
Rounds: 150 Repeats: 10      Seed: 7      Epsilon: 0  
Payoffs: T=5, R=3, P=1, S=0
```

Leaderboard

```
-----  
      Strategy  Mean  
1.   TFT:      2.599, 95% CI [2.441, 2.756]  
2.   GRIM:      2.599, 95% CI [2.441, 2.756]  
3.   PAVLOV:    2.599, 95% CI [2.441, 2.756]  
4.   ALLC:      2.4, 95% CI [2.165, 2.635]  
5.   ALLD:      1.816, 95% CI [1.504, 2.128]
```

Payoff Matrix

```
-----  
      ALLC      ALLD      TFT      GRIM      PAVLOV  
ALLC      3        0        3        3        3  
ALLD      5        1      1.027      1.027      1.027  
TFT       3      0.9933      3        3        3  
GRIM      3      0.9933      3        3        3  
PAVLOV    3      0.9933      3        3        3
```

In a noise-free environment, TFT, GRIM and PAVLOV strategies perform best. This is because they achieve high scores by maintaining mutual cooperation against similar strategies, while being able to retaliate against defectors. In this environment, the only defecting strategy is ALLD. TFT, GRIM and PAVLOV initially cooperate, receiving the sucker payoff. In the next round, they all retaliate and consistently score the punishment payoff, which is the best possible result against ALLD.

Q2: Reciprocity under noise

Noise sweep arguments:

```
// Epsilon = 0.0
--rounds 150 --repeats 30 --epsilon 0.00 --seed 42 --format csv
--strategies TFT,GRIM,PAVLOV,CONTRITE

// Epsilon = 0.05
--rounds 150 --repeats 30 --epsilon 0.05 --seed 42 --format csv
--strategies TFT,GRIM,PAVLOV,CONTRITE

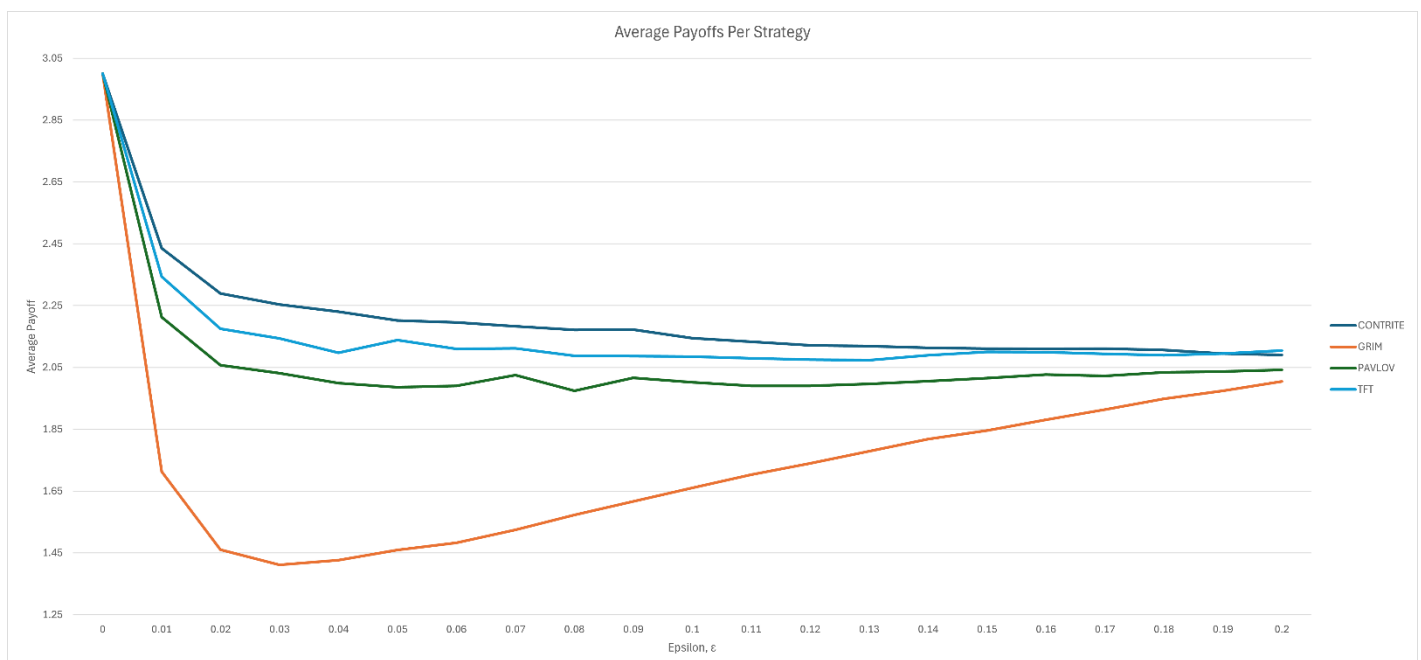
// Epsilon = 0.1
--rounds 150 --repeats 30 --epsilon 0.1 --seed 42 --format csv
--strategies TFT,GRIM,PAVLOV,CONTRITE

// Epsilon = 0.15
--rounds 150 --repeats 30 --epsilon 0.15 --seed 42 --format csv
--strategies TFT,GRIM,PAVLOV,CONTRITE

// Epsilon = 0.2
--rounds 150 --repeats 30 --epsilon 0.2 --seed 42 --format csv
--strategies TFT,GRIM,PAVLOV,CONTRITE
```

The arguments have been summarised for brevity's sake. To produce the graph, I used intervals of 0.01 between each epsilon value and inserted the data from the CSV files generated into Excel.

Results stored in "Noise Sweep Tests" folder.



The graph shows that, without noise, all strategies are mutually cooperative, earning each the reward payoff every round.

As noise is introduced, GRIM collapses at the fastest rate. As it is an unforgiving strategy, once the opponent accidentally swaps to D, GRIM will retaliate and continuously defect. In this particular environment, the other strategies will respond in turn and begin to defect.

In a low-noise environment, GRIM is most likely to earn the punishment payoff, whilst the other strategies may be able to return to mutual cooperation. As noise increases, in cases where GRIM is defecting and the opponent accidentally cooperates, GRIM benefits from the temptation payoff, and in the following round it may return to mutual defection. In contrast, if GRIM accidentally cooperates, all other strategies will respond with C the following round, meaning GRIM will again benefit from the temptation payoff. This can explain how GRIM is able to recover in environments with increased noise.

All other strategies in this environment (PAVLOV, TFT, CTFT) will retaliate but are able to forgive. However, this behaviour is more likely to result in an alternating deadlock. For example, in a match between PAVLOV and TFT, the following sequence can occur:

Round 1: PAVLOV and TFT play C -> CC

Round 2: PAVLOV accidentally defects, TFT mirrors PAVLOV's last C -> DC

Round 3: PAVLOV shifts to C (win-stay/lose-shift logic), TFT mirrors D -> CD

Round 4: PAVLOV shifts to D, TFT mirrors C -> DC

This pattern will continue until one of the strategies is affected by noise. Due to the nature of CTFT, PAVLOV and TFT, this pattern is observable in all matches between each strategy, resulting in frequent temptation and sucker payoffs.

The overall observed decrease in the average payoff can be explained by the increased likelihood that every strategy will be affected by an alternating or defecting deadlock as noise increases.

CTFT is the most resistant to noise, evidenced by it having the highest average payoff per round at most values of epsilon. This is because it is able to detect its own accidental defections and 'apologise', increasing the likelihood that mutual cooperation will occur in all matches involving CTFT, including self-play. It is important to note that CTFT's performance degrades in noisy environments, possibly because it is detecting its own defection more often and therefore frequently receiving the sucker payoff.

Q3: Robustness to exploiters

To best understand the performance of exploitative strategies in particular matches, I have presented the results using the text file format. The leaderboard allows us to understand how each strategy performs as an average of all their matches. The payoff matrix displays the performance of each strategy within each of its matches.

All results for the tournaments run are stored in the “results/Exploiters Tests” folder.

“Exploiters (noise-free)” arguments:

```
--rounds 150 --repeats 10 --seed 7 --format text
--strategies ALLC,TFT,CONTRITE,PAVLOV,PROBER,ALLD,FBF
```

Tournament Summary

```
-----
Rounds: 150 Repeats: 10 Seed: 7      Epsilon: 0
Payoffs: T=5, R=3, P=1, S=0
```

Leaderboard

```
-----
Strategy Mean
1. PROBER: 2.985, 95% CI [2.809, 3.161]
2. TFT: 2.712, 95% CI [2.596, 2.829]
3. CONTRITE: 2.712, 95% CI [2.596, 2.829]
4. PAVLOV: 2.712, 95% CI [2.596, 2.829]
5. FBF: 2.711, 95% CI [2.595, 2.828]
6. ALLC: 2.151, 95% CI [1.929, 2.374]
7. ALLD: 1.598, 95% CI [1.368, 1.828]
```

Payoff Matrix

```
-----
ALLC TFT CONTRITE PAVLOV PROBER ALLD FBF
ALLC 3 3 3 3 0.06 0 3
TFT 3 3 3 3 2.993 0.9933 3
CONTRITE 3 3 3 3 2.993 0.9933 3
PAVLOV 3 3 3 3 2.993 0.9933 3
PROBER 4.96 2.993 2.993 2.993 2.987 0.98 2.987
ALLD 5 1.027 1.027 1.027 1.08 1 1.027
FBF 3 3 3 3 2.987 0.9933 3
```

As an exploitative strategy in an environment without noise, ALLD performs best against ALLC as it is the only strategy not capable of retaliation. Otherwise, it is most likely to score the punishment payoff.

Furthermore, PROBER successfully identifies which strategies can be exploited and uses this to its advantage to maximise its output in all matches. I have used the standard probing sequence in my implantation, {C, D, C, C}.

In a match against ALLC, it is expected that ALLC will ignore PROBER’s defection in round 2 and continue to cooperate. Its only opportunities to score are during the probing phase, where PROBER responds with C. This is reflected in the payoff matrix, as ALLC’s average payoff against PROBER is very low, but not exactly zero (sucker payoff).

TFT is expected to mirror all the opponent’s previous responses. Therefore, against PROBER, it will mirror the defection from round two in the next round. At this point, PROBER identifies that TFT cannot be exploited. It is important that PROBER includes a fourth round in its probing phase, to avoid misalignment with reactive strategies. TFT will mirror the cooperation from

round three, so both strategies mutually cooperate in round four and will continue to do so for the remainder of the match. This is reflected in the payoff matrix as PROBER and TFT have identical averages, but they are not equal to three (reward payoff).

I have also included my own strategy, FBF. It resists exploitation similar to TFT, but it performs worse in a match against PROBER. The following sequence demonstrates why this occurs:

Round 1: PROBER and FBF play C -> CC
 Round 2: PROBER defects, FBF cooperates by default -> DC
 Round 3: PROBER cooperates, FBF punishes for 2 rounds -> CD
 Round 4: PROBER identifies it can't exploit so cooperates, FBF punishes -> CD
 Round 5: PROBER behaves like TFT so defects, FBF tests for cooperation -> DC
 Round 6: PROBER mirrors cooperation, FBF acknowledges PROBER can cooperate so continues to play C -> CC

Both strategies continue to cooperate for the remainder of the match. However, because it took longer to resolve the defection, they both receive smaller payoffs, compared to the PROBER vs TFT match.

“Exploiters (with noise, $\epsilon=0.05$)” arguments:

```
--rounds 150 --repeats 10 --seed 7 --epsilon 0.05 --format text
--strategies ALLC,TFT,CONTRITE,PAVLOV,PROBER,ALLD,FBF
```

Tournament Summary

```
-----
Rounds: 150 Repeats: 10 Seed: 7      Epsilon: 0.05
Payoffs: T=5, R=3, P=1, S=0
```

Leaderboard

```
-----
Strategy  Mean
1.  FBF:      2.456, 95% CI [2.341, 2.571]
2.  PROBER:   2.432, 95% CI [2.261, 2.603]
3.  CONTRITE: 2.381, 95% CI [2.271, 2.491]
4.  TFT:      2.337, 95% CI [2.236, 2.438]
5.  PAVLOV:   2.298, 95% CI [2.191, 2.404]
6.  ALLC:     2.134, 95% CI [1.941, 2.327]
7.  ALLD:     1.943, 95% CI [1.75, 2.137]
```

Payoff Matrix

```
-----
      ALLC      TFT      CONTRITE  PAVLOV  PROBER  ALLD      FBF
ALLC      2.952      2.813      2.881      2.659      0.5343      0.1987      2.9
TFT        3.04      2.273      2.525      2.349      2.258      1.091      2.823
CONTRITE   2.986      2.469      2.802      2.351      2.085      1.114      2.861
PAVLOV     3.127      2.341      2.43      2.065      2.265      1.033      2.823
PROBER     4.491      2.26      2.215      2.28      1.942      1.082      2.753
ALLD       4.704      1.331      1.364      1.543      1.384      1.163      2.115
FBF        2.97      2.648      2.806      2.56      2.487      0.8803      2.838
```

In the same environment with slightly more noise, it is expected that most strategies will continue to resist exploitation. However, their behaviours are now less predictable, and it is possible for all strategies to receive any payout in all of their matches. This results in ALLD's slightly improved performance, as mutual punishment is no longer guaranteed.

Equally as expected, PROBER performs worse overall because it cannot reliably detect which strategies it can exploit due to noise.

On average, PROBER continues to exploit ALLC when noise is introduced, but this may have not been the case in every match. If ALLC had defected during the probing phase, it would incorrectly identify that it cannot exploit the opponent, reducing its average payoff. In addition to this, if the probing phase executed as expected, the following outcomes are still possible:

PROBER exploits correctly, ALLC defects due to noise -> DD (punishment payoff)

PROBER cooperates due to noise, ALLC cooperates -> CC (reward payoff)

PROBER cooperates, ALLC defects -> CD (sucker payoff)

As the chances of either strategy's response being affected by noise are low, these outcomes are less likely to occur, and as a result, do not have a great impact on the average payoff.

PROBER could incorrectly identify TFT as an exploitable opponent, in which case, it would be more likely that both strategies receive the punishment payoff. Even if this is not the case, it is possible to break the cycle of mutual cooperation if either strategy accidentally defects.

Similar to TFT, in a match between PROBER and FBF, either strategy can defect during mutual cooperation. However, FBF can resolve situations like this, as it tests for cooperation after punishing the opponent, eventually returning to mutually cooperation. Despite this, FBF performs worse than PROBER in their match. This could be a result of FBF's trusting design. This is beneficial for both strategies long term as they return to mutual cooperation more often, but PROBER will benefit from the temptation payoff in the process, as FBF must play C while PROBER is still mirroring the previous punishment.

“Exploiters (with noise, $\epsilon=0.2$)” arguments:

```
--rounds 150 --repeats 10 --seed 7 --epsilon 0.2 --format text
--strategies ALLC,TFT,CONTRITE,PAVLOV,PROBER,ALLD,FBF
```

Tournament Summary

Rounds: 150 Repeats: 10 Seed: 7 Epsilon: 0.2
Payoffs: T=5, R=3, P=1, S=0

Leaderboard

Strategy Mean
1. ALLD: 2.328, 95% CI [2.217, 2.44]
2. PROBER: 2.28, 95% CI [2.179, 2.381]
3. TFT: 2.251, 95% CI [2.175, 2.327]
4. CONTRITE: 2.212, 95% CI [2.134, 2.29]
5. FBF: 2.207, 95% CI [2.12, 2.294]
6. PAVLOV: 2.201, 95% CI [2.112, 2.291]
7. ALLC: 2.066, 95% CI [1.955, 2.177]

Payoff Matrix

ALLC TFT CONTRITE PAVLOV PROBER ALLD FBF
ALLC 2.778 2.368 2.54 2.104 1.568 0.85 2.256
TFT 2.973 2.295 2.441 2.262 2.172 1.386 2.227
CONTRITE 2.903 2.292 2.434 2.242 2.038 1.372 2.202
PAVLOV 3.122 2.27 2.385 2.106 1.942 1.328 2.256
PROBER 3.411 2.26 2.391 2.292 1.917 1.412 2.279
ALLD 3.833 2.04 2.099 2.319 2.042 1.599 2.366
FBF 3.064 2.228 2.395 2.235 2.017 1.283 2.226

The output from a tournament running in an environment with even greater noise shows that performance of all strategies begins to collapse. The variance between the average payoffs per round for each strategy is smaller, indicating that noise may have had a greater impact in each match. I have run the same tournament again but only changing the seed to demonstrate the magnitude of this impact.

```
--rounds 150 --repeats 10 --seed 87 --epsilon 0.2 --format text
--strategies ALLC,TFT,CONTRITE,PAVLOV,PROBER,ALLD,FBF
```

Tournament Summary

Rounds: 150 Repeats: 10 Seed: 87 Epsilon: 0.2
Payoffs: T=5, R=3, P=1, S=0

Leaderboard

Strategy Mean
1. ALLD: 2.308, 95% CI [2.194, 2.422]
2. PROBER: 2.292, 95% CI [2.189, 2.395]
3. FBF: 2.253, 95% CI [2.169, 2.338]
4. TFT: 2.24, 95% CI [2.167, 2.314]
5. PAVLOV: 2.238, 95% CI [2.154, 2.321]
6. CONTRITE: 2.226, 95% CI [2.149, 2.303]
7. ALLC: 2.068, 95% CI [1.952, 2.184]

Payoff Matrix

ALLC TFT CONTRITE PAVLOV PROBER ALLD FBF
ALLC 2.778 2.427 2.549 2.245 1.48 0.834 2.165
TFT 2.943 2.26 2.436 2.258 2.114 1.449 2.22
CONTRITE 2.864 2.306 2.46 2.274 2.085 1.362 2.234
PAVLOV 3.068 2.233 2.482 2.18 2.109 1.338 2.252
PROBER 3.482 2.174 2.36 2.268 2.039 1.405 2.317
ALLD 3.867 2.009 2.13 2.23 2.066 1.553 2.301
FBF 3.102 2.234 2.455 2.242 2.129 1.349 2.263

Under noise, it is more likely that PROBER identifies an exploitable opponent during the probing phase. ALLD will always choose to defect, regardless of its opponent. This means it is more likely that both strategies rely on consistently choosing to defect, but it is important to note that this choice is also more likely to be affected by noise.

On the other hand, reactive strategies rely on the responses of their opponent. As a result, it is likely that it may choose to cooperate the round after an exploitative opponent accidentally swaps to C. In which case, the exploitative strategy will benefit.

While it is true that noise can affect both opponents simultaneously, sometimes resulting in exploitative strategies receiving the sucker payoff, it remains likely that ALLD and PROBER will defect the following round, earning either the temptation or punishment payoffs.

We can conclude that ALLD suffers in an environment without noise, where strategies are able to resist exploitation, as it is unable to adapt to its opponent. In the same environment, PROBER will likely perform best, as it is able to maximise its output against strategies that can and cannot resist exploitation.

However, as noise is introduced, strategies begin to make more mistakes in their logic, allowing exploitative strategies that consistently choose to defect to benefit.

Q4: Evolutionary Selection

Evolutionary tournaments show which strategies perform best amongst the other strategies within their population. The populations of each strategy remain stable as long as they all cooperate with each other.

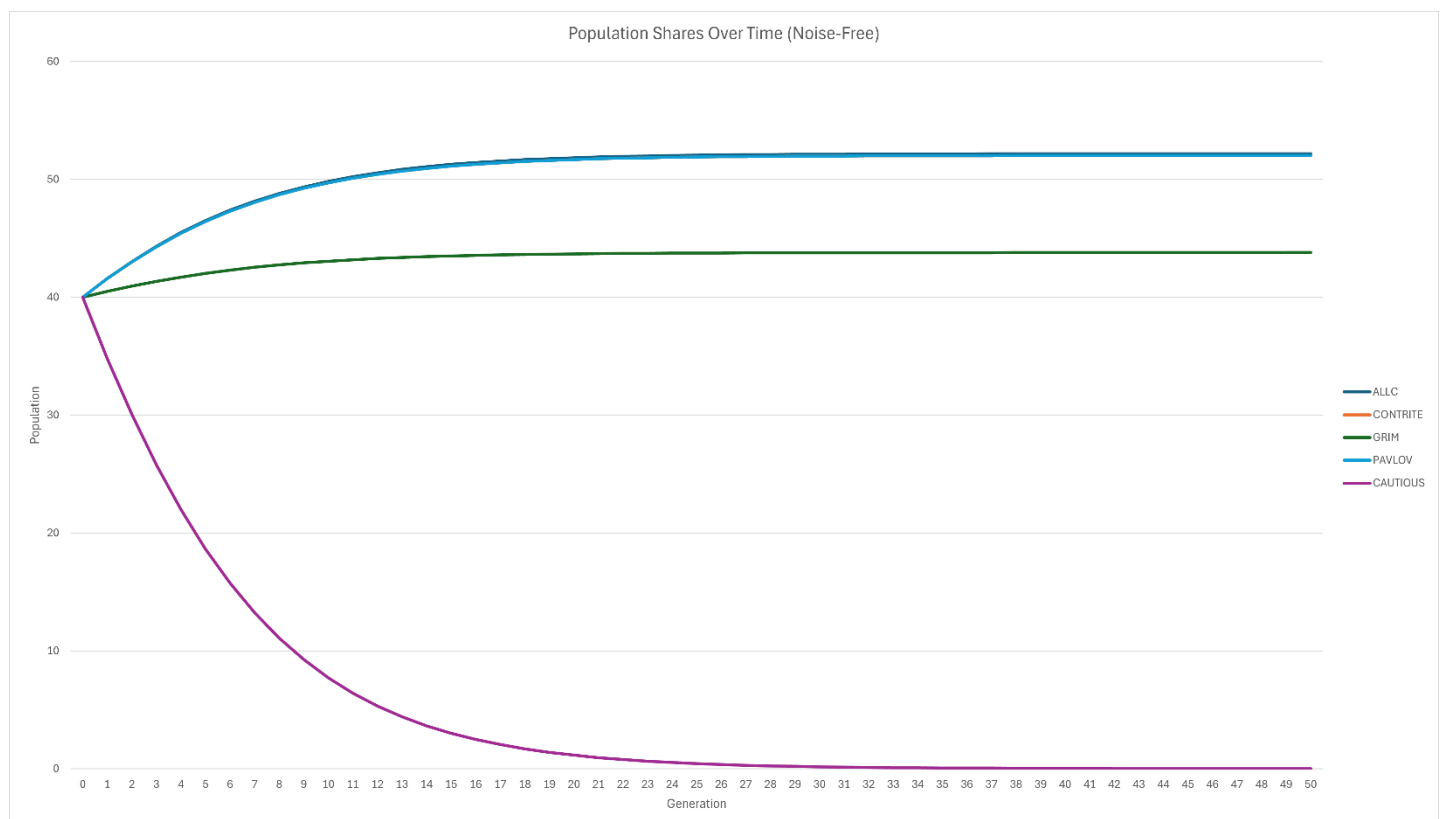
Initially all strategies represent an equal share of the total population and after each generation, the population is updated to increase the number of individual members using the strategy with the highest average payoff.

To demonstrate my implementation of evolutionary tournaments, I have chosen to use ALLC, CTFT, GRIM, PAVLOV and my own strategy CAUTIOUS. ALLC is consistently cooperative, and CTFT ‘apologises’ for accidentally defecting because of noise. GRIM is initially cooperative, but unforgiving once the opponent defects, and PAVLOV relies on ‘win-stay/lose-shift’ logic. CAUTIOUS operates as a tentative version of PAVLOV, in which it is initially suspicious of the opponent, but attempts to improve its payoff by signalling for mutual cooperation. With this particular selection, we can observe the benefits of mutual cooperation and how well each strategy adapts to the presence of noise.

All results for the tournaments run are stored in the “results/evolutionTests” folder.

“Evolutionary tournament (noise-free)” arguments:

```
--rounds 150 --repeats 10 --seed 7 --evolve --format csv
--population 200 --generations 50
--strategies ALLC,ALLD,CONTRITE,PAVLOV,GRIM
```



The graph above clearly demonstrates that some strategies benefit from mutual cooperation more than others. In particular, CAUTIOUS is extinct by the final generation. Its performance is mostly hindered by strategies that can retaliate, especially GRIM.

Against GRIM, CAUTIOUS will observe the following sequence:

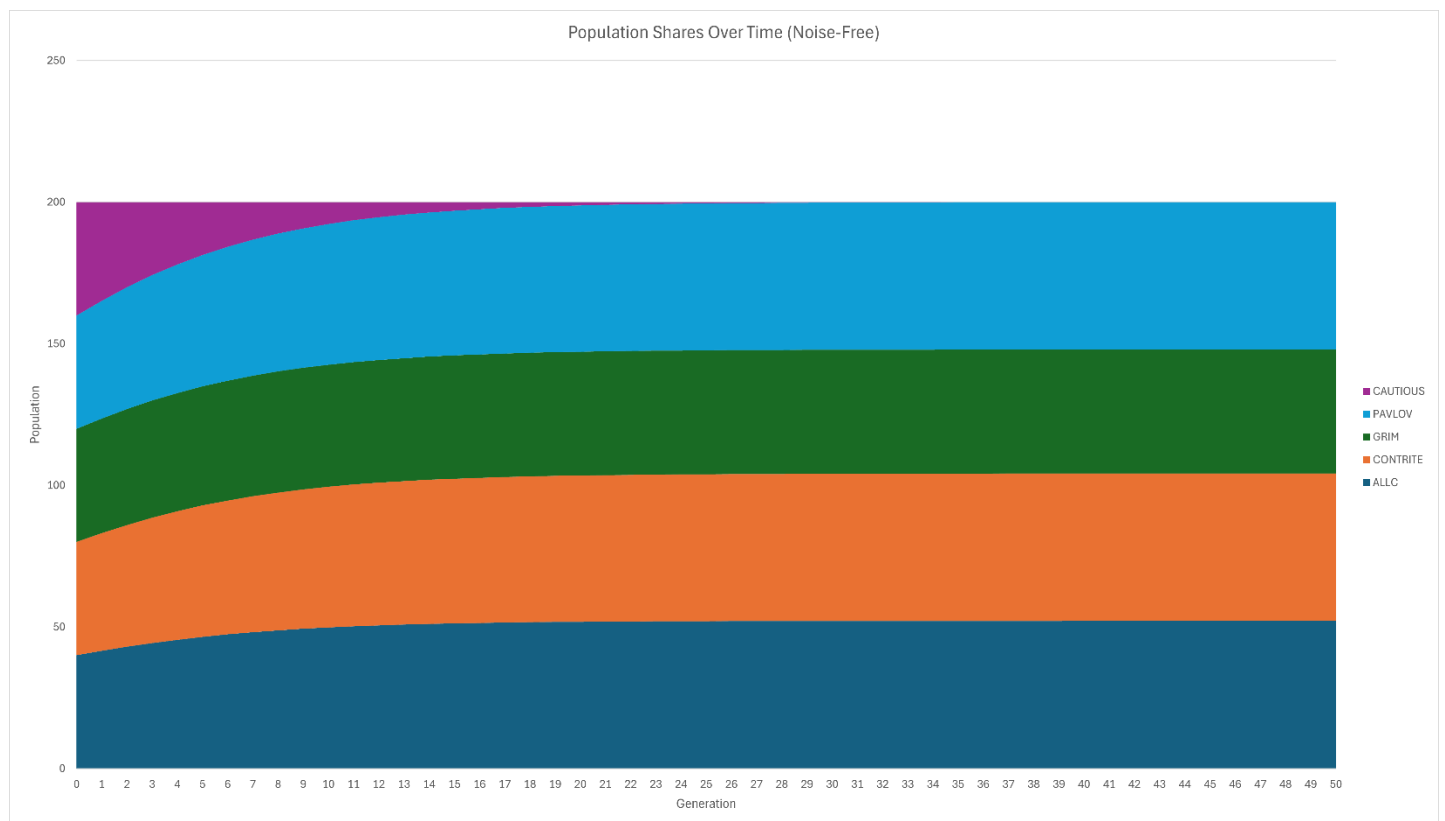
- Round 1: CAUTIOUS plays D, GRIM plays C -> DC
- Round 2: CAUTIOUS plays D, GRIM detects defection and will only play D -> DD
- Round 3: CAUTIOUS uses 'win-stay/lose-shift' logic and plays D, GRIM plays D -> DD
- Round 4: -> DD
- Round 5: CAUTIOUS attempts to reconcile and plays C -> CD
- Round 6: -> CD
- Round 7: Reconciliation failed, so CAUTIOUS returns to 'win-stay/lose-shift' -> DD

Every three rounds against GRIM, CAUTIOUS will always try to reconcile for defecting, but it always fails. In attempting to do so, it scores the sucker payoff for two rounds, significantly decreasing its average payoff.

Against PAVLOV and CTFT, after two rounds of defecting, CAUTIOUS is able to successfully reconcile and both strategies mutually cooperate. Despite the initial defection from CAUTIOUS, ALLC will always choose to cooperate, and to follow the 'win-stay/lose-shift' logic, CAUTIOUS will follow suit.

Overall, CAUTIOUS has the smallest average payoff after each generation, but because it is able to cooperate with PAVLOV, CTFT and ALLC, its extinction is not immediate.

As multiple lines overlap on the line graph, I have also included a stacked area graph for clarity.



With this, we can see that it was ALLC, PAVLOV and CTFT that had equal shares in the final population. GRIM did not become extinct but represented a smaller proportion.

Although all four of these strategies are mutually cooperative in their matches against each other, GRIM represents a smaller share due to the inclusion of CAUTIOUS.

GRIM is the strategy that is most disadvantaged by the presence of CAUTIOUS, as the matches between the two strategies decrease its average payoff. Despite this, GRIM always has the better average payoff after each generation, because it is able to mutually cooperate with the other strategies every round. This explains why it remains present in the final population.

Matches involving PAVLOV and CTFT, against similar cooperative strategies, are also earning a reward payoff. Against CAUTIOUS, it will eventually reach the same result, but it must go through three rounds of mutual defection.

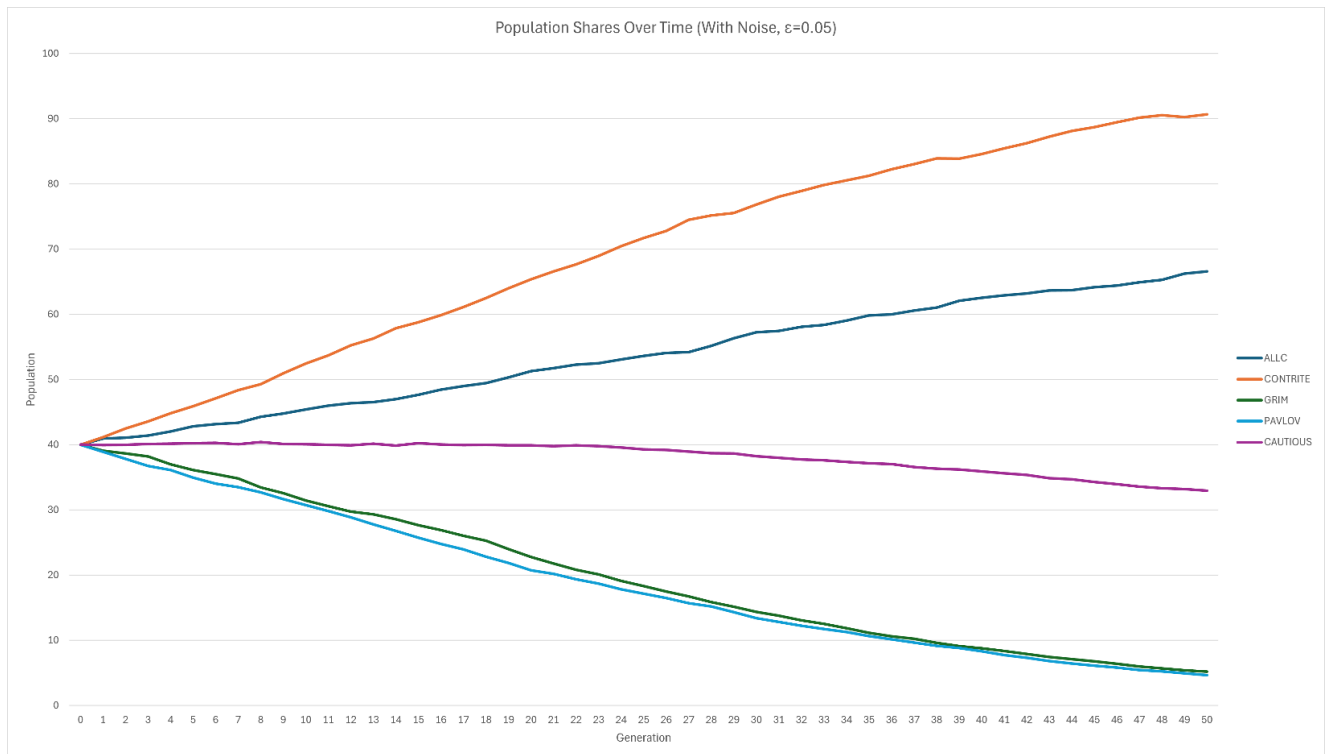
The same is true of ALLC, but it must only experience two rounds where CAUTIOUS is defecting, before it begins to cooperate. As all matches involving ALLC result in mutual cooperation, it maximises its average payoff after each generation. As the share of individuals using the CAUTIOUS strategy decreases, ALLC experiences fewer rounds where it receives the sucker payoff.

Once CAUTIOUS becomes extinct, all remaining strategies are mutually cooperative, so the population remains stable.

Overall, the cooperative strategies dominate in this environment because they hold the majority. However, cooperative strategies that are exploited, will have a lower average and, therefore, may become extinct.

“Evolutionary tournament (with noise, $\epsilon=0.05$)” arguments:

```
--rounds 150 --repeats 10 --seed 7 --evolve --format csv
--population 200 --generations 50 --epsilon 0.05
--strategies ALLC, ALLD, CONTRITE, PAVLOV, GRIM
```



In addition to highlighting the most compatible strategies in an environment, evolutionary tournaments excel at presenting the strategies that are most affected by noise. This graph demonstrates this for the same set of strategies as the tournament described above.

With noise present, the final population shares vary greatly to the same environment without noise. PAVLOV and GRIM now represent the smallest portions of the total population.

All matches involving GRIM are likely to result in mutual defection. Against PAVLOV or CTFT, once either strategy defects, GRIM will no longer cooperate, bringing its average closer to the value of the punishment payoff.

In a match between GRIM and CAUTIOUS, CAUTIOUS is mostly unaffected by noise. By the second round, GRIM is already unforgiving of its defection, so it is more like that it will choose D. If the match is stuck following the sequence as described in the example given earlier, if GRIM defected, CAUTIOUS would change to C but continue to defect the following round as long as GRIM's response is unaffected by noise. Considering the reverse situation, where CAUTIOUS defects, GRIM is unaffected.

The number of matches involving GRIM cooperating with its opponent, have significantly decreased with the introduction of noise, resulting in its lower average payoff after each generation and its reduced presence in the population.

PAVLOV always aims to mutually cooperate or defect with their opponents, depending on their responses. Under noise, it is likely that against this particular set of strategies, it will begin to bounce between mutually cooperating and mutually defecting.

Against CTFT, if already cooperating, once PAVLOV defects due to noise, CTFT will mirror this, and both will mutually punish. The reverse occurs when both strategies are already punishing and PAVLOV defects.

The graph also shows that PAVLOV, although marginally, has a greater population than GRIM. This may be a result of its matches against CAUTIOUS and ALLC. Despite the effects of noise, ALLC will always choose to cooperate, so it is more likely that PAVLOV will return to cooperation. Against CAUTIOUS, if both strategies are stuck in a punishing loop for three rounds, CAUTIOUS will reconcile and both will eventually return to cooperating. This occurs whether either strategy defects.

As ALLC is most likely to cooperate, in many of its matches it will continue to perform well in a noisy environment. Against CAUTIOUS, if either strategy defects, it is unlikely that it will defect multiple rounds in a row, meaning will eventually return to cooperating. The same is true in matches against PAVLOV and CTFT.

In matches against GRIM, ALLC may experience a lower average payoff because GRIM is unforgiving of its accidental defections. However, due to the presence of cooperative strategies, the population of ALLC increases as GRIM's population decreases.

CAUTIOUS is able to sustain a larger population in a noisy environment because, as described before, it is mostly hindered by GRIM. As the number of members using the GRIM strategy fall, it experiences more matches where it is mutually cooperating. In addition, it outperforms PAVLOV because it is able to resolve loops of mutual punishments.

From the gathered results, we can determine that CTFT is the most unaffected by noise. This is because it apologises for defecting when it was originally cooperating with the opponent, by cooperating for two rounds, similar to CAUTIOUS. The reason it outperforms all other strategies is because it recovers from its mistakes quickly. Furthermore, GRIM is the only strategy that is likely to defect regardless of CTFT's response, but because it represents a small minority of the total population, it is phased out, allowing the population of CTFT to grow even more.

We can conclude that, in an environment including strategies capable of cooperating, evolutionary tournaments promote the presence of those particular strategies, punishing defection. However, I would suspect that if a strategy that is able to exploit and cooperate was included, that would dominate the tournament instead, as it would be able to maximise its output in every match. Furthermore, CTFT can dominate in environments with and without noise.

In an extension of this investigation, I would run more evolutionary tournaments involving exploitative strategies to understand how they would perform in such an environment. I would also experiment with varying initial population shares.

Q5: Strategic Complexity Budget (SCB) Exploration

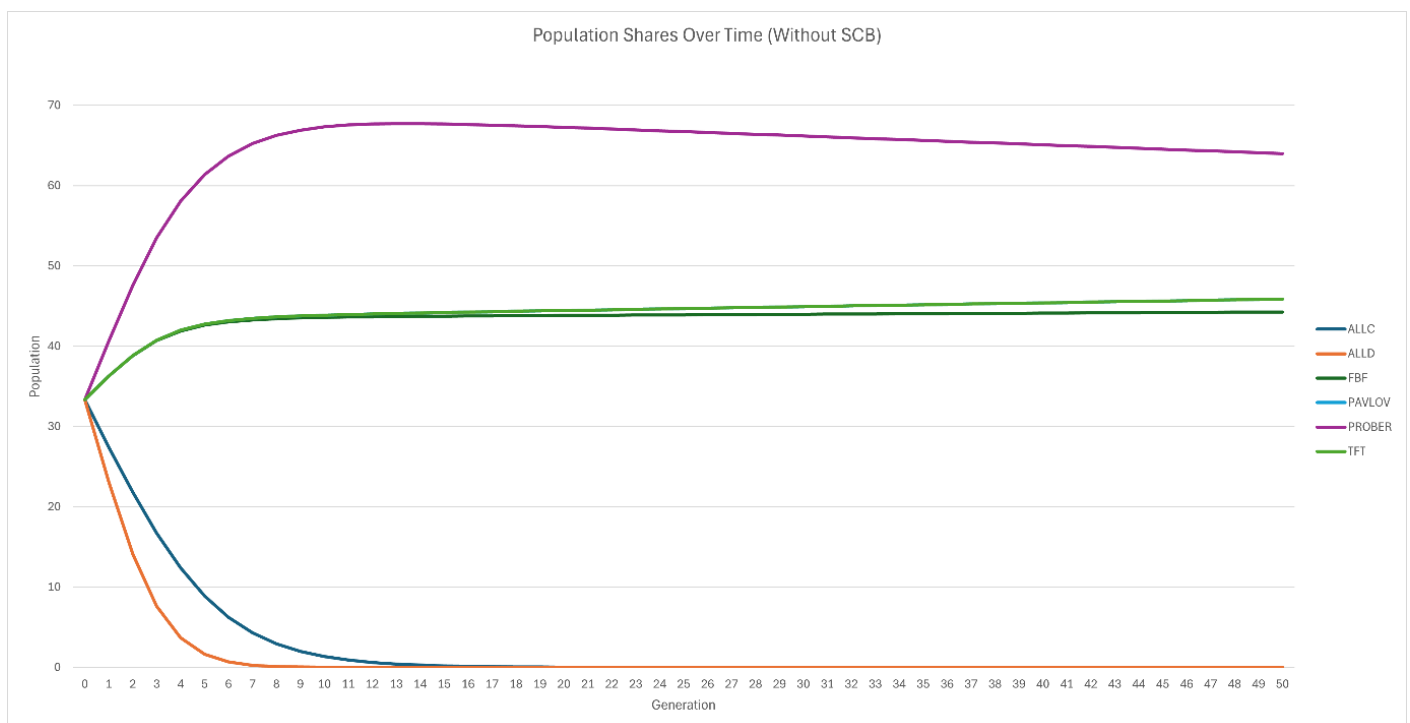
Strategic Complexity Budget (SCB) used in this implementation:

- ALLC, ALLD, RND0.3 = 1 (No memory)
- PAVLOV, GRIM, TFT = 2 (Refer to the opponents' previous response)
- CAUTIOUS, CONTRITE, FBF, PROBER = 3 (Extra logic for noise, probing, identification of patterns and punishing)

All results for the tournaments run are stored in the “results/SCB Tests” folder.

“Evolutionary tournament (without SCB)” arguments:

```
--rounds 150 --repeats 10 --seed 142 --format csv
--evolve --population 200 --generations 50
--strategies ALLC, PROBER, PAVLOV, ALLD, FBF, TFT
```



In order to evaluate the effects of SCB, I have first run an evolutionary tournament without it as a baseline. I have used ALLC, ALLD, PAVLOV, TFT, PROBER and FBF. I intentionally chose to include two strategies from each complexity group to investigate the bearing complexity has on the evolutionary outcomes.

Before discussing the results from the tournament run with SCB, it is important to identify the dominating strategies in this tournament.

As evidenced by the graph, we can see that PROBER excels representing the majority of the population, whilst ALLD and ALLC become extinct.

Q4 shows evidence that cooperative strategies, like ALLC, perform well in evolutionary tournaments. However, this data disputes that, largely due to the presence of PROBER. As an exploitative strategy that is capable of cooperation, PROBER exploits ALLC, reducing its average

payoff. Moreover, in matches against the other strategies in the environment, it is expected that PROBER will mutually cooperate. This means that in all matches involving PROBER, it is able to maximise its output.

ALLD becomes extinct as it benefits most from defecting against FBF and ALLC in the entire population. In matches against other strategies, ALLD is more likely to receive the punishment payoff, decreasing its average payoff. As a result, its population share decreases over time.

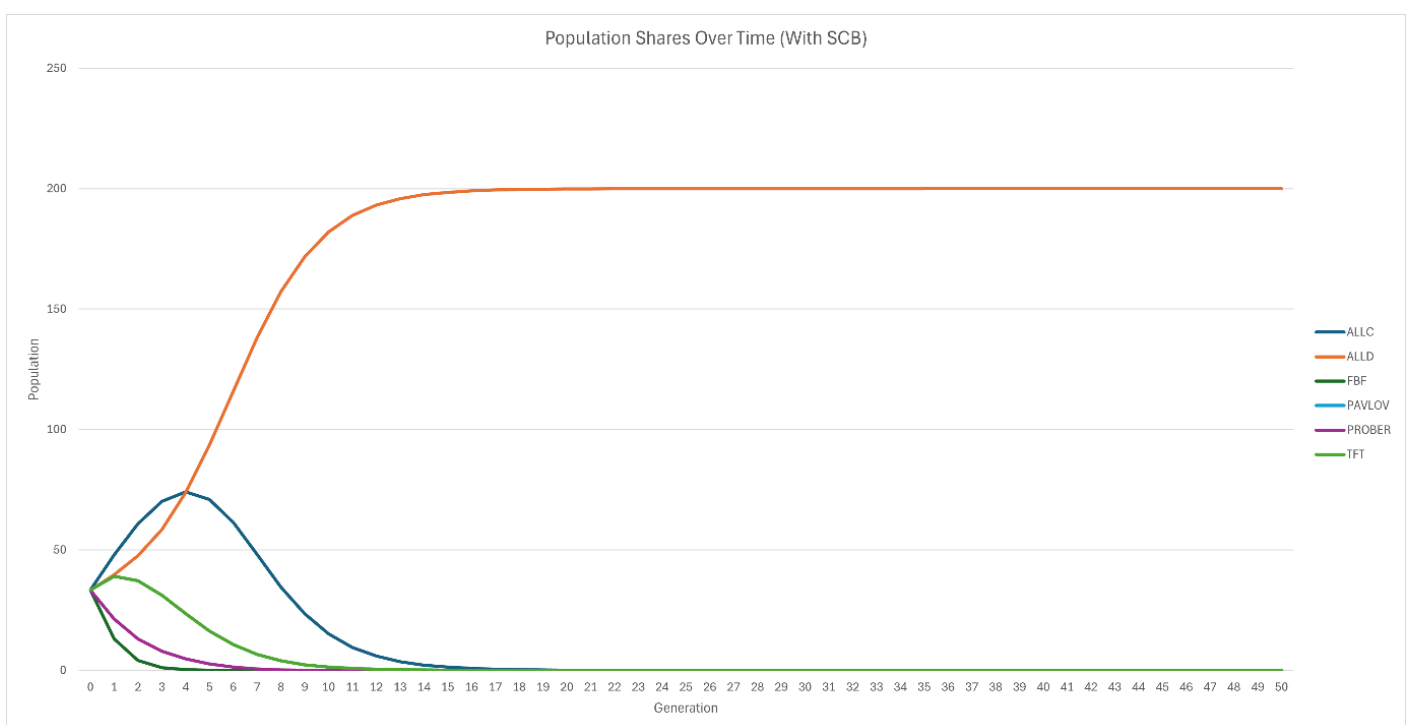
ALLC also becomes extinct in this tournament because it is exploited by ALLD and PROBER. As PROBER greatly excels in this environment, its population grows at the fastest rate. Therefore, ALLC must face PROBER more often, decreasing the average payoff, and in turn its population.

FBF and TFT maintain stable populations, but do not perform as well as PROBER. These strategies are most likely to achieve the reward payoff after mutually cooperating with their opponents. Their averages are only negatively affected from matches against ALLD, where they are most likely to achieve the punishment payoffs.

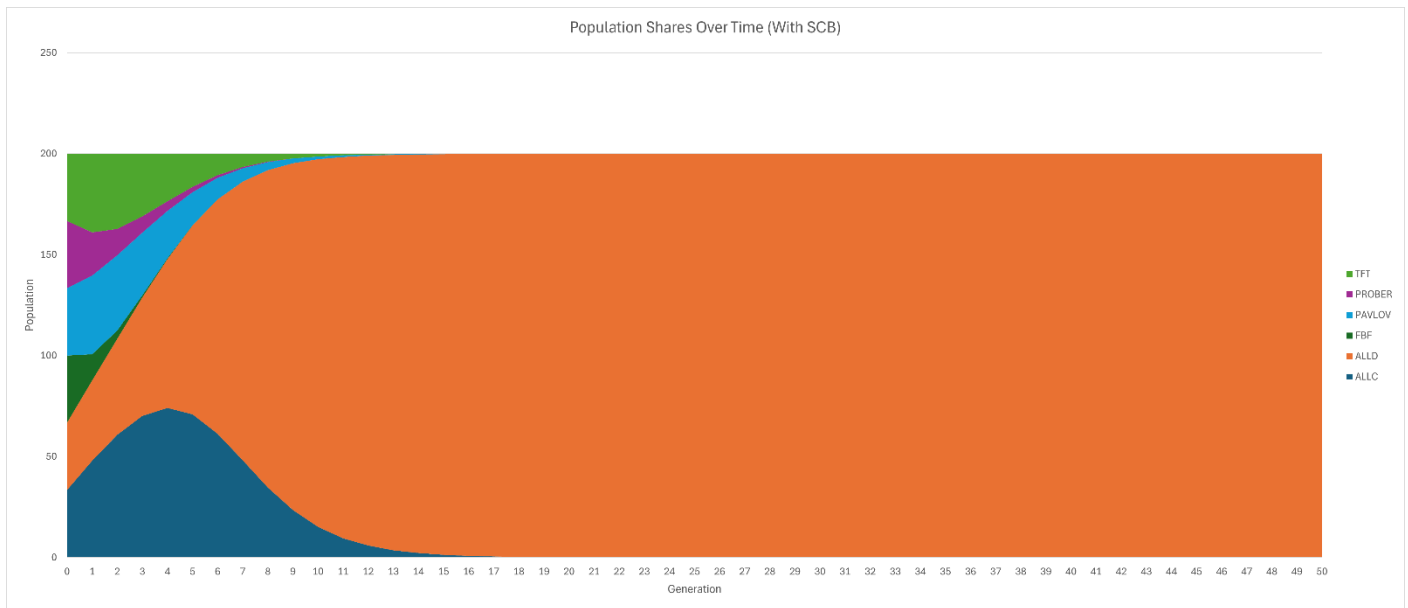
To summarise, evolutionary tournaments encourage both exploitation and cooperation, as this maximises the average payoff. The only strategy capable of this is PROBER, and therefore, benefits most.

“Evolutionary tournament (with SCB)” arguments:

```
--rounds 150 --repeats 10 --seed 142 --format csv
--evolve --population 200 --generations 50
--strategies ALLC, PROBER, PAVLOV, ALLD, FBF, TFT --scb
```



As some of the lines overlap, I have included a stacked area graph to better represent change in shares.



When SCB is incorporated, as expected, the more complex strategies are penalised. In the same environment in which PROBER previously excelled, it now quickly becomes extinct. The only strategies that are able to grow under these conditions are ALLC, ALLD and, momentarily, TFT.

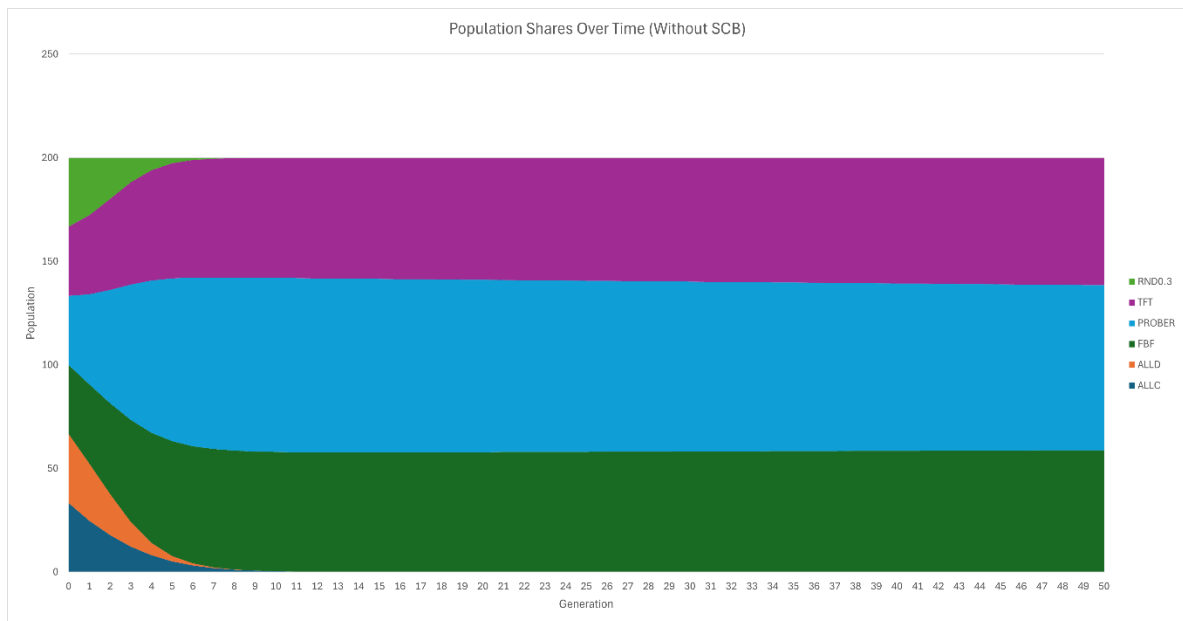
As the simplest strategies in this population, ALLC and ALLD grow at the fastest rates. Although TFT is not as simple, it is able to grow for a single generation as it retaliates well against ALLD and cooperates with the other strategies. PAVLOV is equally as complex as TFT, and it does not experience as harsh of a penalty as the complex strategies. This partly explains why its rate of extinction is slower. In combination with this, after each generation there are fewer cooperating strategies and more members using ALLD. This results in a lower average payoff for both PAVLOV and TFT, as the best they can do against ALLD is mutually defect. Eventually, they are phased out as they are dominated by the simpler strategies.

Out of the cooperating strategies, ALLC is able to grow the most. However, as the more complex cooperating strategies become extinct, it is exploited by ALLD more frequently, resulting in its own extinction.

To further investigate the effects of SCB, I have run this tournament again, with and without SCB applied, but rather than including PAVLOV, I have included RND0.3, as it is the other strategy I have defined as being simple.

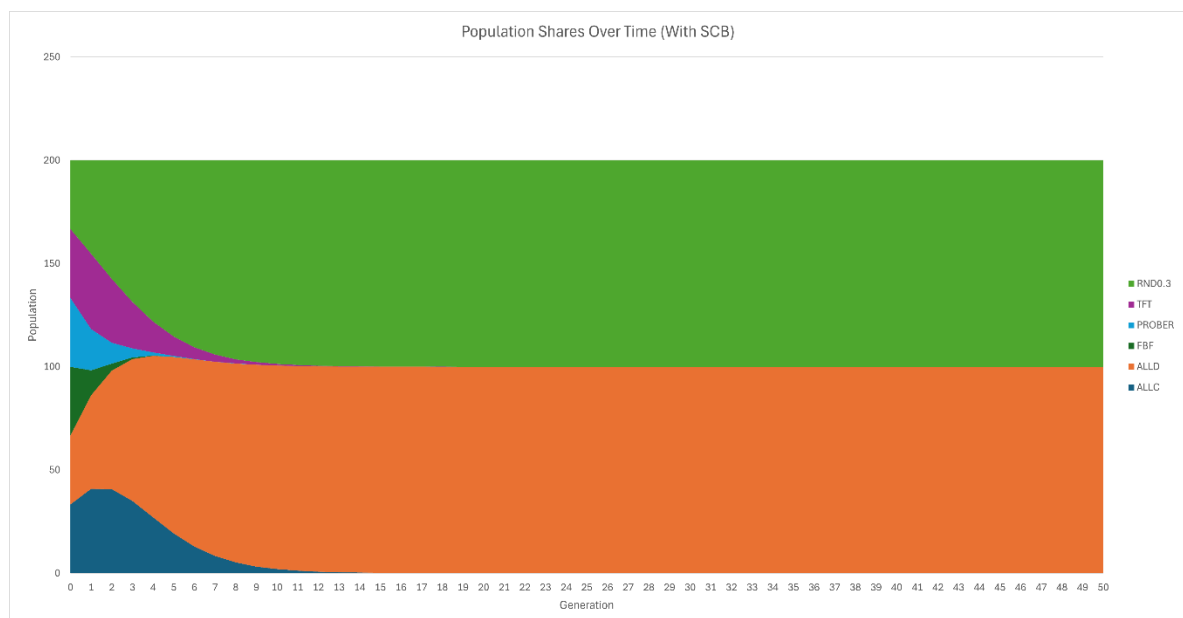
“Evolutionary tournament (without SCB)” arguments:

```
--rounds 150 --repeats 10 --seed 142 --format csv  
--evolve --population 200 --generations 50  
--strategies ALLC, PROBER, RND0.3, ALLD, FBF, TFT
```



“Evolutionary tournament (with SCB)” arguments:

```
--rounds 150 --repeats 10 --seed 142 --format csv  
--evolve --population 200 --generations 50  
--strategies ALLC, PROBER, RND0.3, ALLD, FBF, TFT --scb
```



Here, we can see that ALLD is no longer allowed to dominate, because it cannot exploit RND0.3, due to its random nature. This tournament better highlights that SCB allows simple strategies to dominate, even in strategies with more robust logic.

To conclude, we can deduce that with SCB applied, simpler strategies can dominate in an environment where they may have become extinct without it. However, when in use, it is important to consider the compatibility of strategies – if one simple strategy can exploit another, that strategy will dominate the tournament.